Task Management System

Project Overview

Develop a modern and responsive **Task Management System frontend** using Next.js.

The application will allow users to login, view and manage their tasks, update task status, search and filter tasks, and view task details.

Admins will have a separate dashboard where they can manage users, tasks, assign tasks, and view overall task statistics.

The frontend should consume the provided REST APIs and provide a clean, responsive and user-friendly interface.

Technology Stack

Use the following technologies:

- Next.js
- React.js
- TypeScript
- Tailwind CSS
- REST APIs
- Axios or Fetch API
- React Hook Form or equivalent form library
- Appropriate state management solution where required

Use reusable components and maintain a clean project structure.

Main Features

1. Authentication

Create authentication screens for:

Login

The login page should contain:

- Email
- Password
- Login button
- Validation messages
- Loading state
- Error handling

After successful login, the user should be redirected to the appropriate dashboard.

Registration

Create a registration page containing:

- Name
- Email
- Password
- Confirm password
- Register button
- Validation

2. Protected Routes

Authenticated users should be able to access the application dashboard.

Unauthenticated users should not be able to access protected pages.

Example:

```
Not Logged In
      ↓
Login Page

Logged In
      ↓
Dashboard
```

Admin-only pages should not be accessible to normal users.

3. User Dashboard

Create a dashboard for normal users.

The dashboard should show:

- Total assigned tasks
- Pending tasks
- In-progress tasks
- Completed tasks
- High-priority tasks

Also show a list of recent tasks.

Example:

```
Dashboard
--------------------------------
Total Tasks       12
Pending            5
In Progress        3
Completed          4
--------------------------------

Recent Tasks
```

```
--------------------------------
Complete API        High      Pending
Frontend UI         Medium    In Progress
Testing             Low       Completed
```

The dashboard data should come from the backend APIs.

4. Task Management

Create a task management section where users can view their tasks.

Each task should display:

- Task title
- Description
- Priority
- Status
- Due date
- Assigned user
- Created date

Users should be able to open a task to view its complete details.

5. Task Details

Create a dedicated task details page.

The page should display complete task information.

Example:

```
Task Details

Title:
```

```
Complete Backend API

Description:
Develop the authentication module.

Priority:
High

Status:
In Progress

Due Date:
25 September 2026

Assigned To:
Rahul Sharma

Created By:
Admin
```

The user should be able to update the task status where permitted.

6. Create / Edit Task

Provide a task form for creating and editing tasks.

Fields:

- Title
- Description
- Priority
- Status
- Due date
- Assigned user

The form should have:

- Client-side validation
- Loading state
- Success message
- Error message
- Proper API integration

7. Task Assignment

Admin should be able to assign tasks to users.

The task creation/edit form should contain an **Assign User** field.

Example:

```
Create Task

Title:       [_________________]

Description: [_________________]

Priority:    [High ▼]

Due Date:    [25/09/2026]

Assign To:   [Rahul Sharma ▼]

             [Create Task]
```

After assigning the task, the backend will send an email notification to the assigned user's email address.

The frontend should show an appropriate success message after the task is created successfully.

8. Task Search & Filtering

The task listing page should provide:

Search

Search tasks by title.

Filters

Filter tasks by:

- Status
- Priority
- Assigned user

Sorting

Allow sorting by:

- Due date
- Created date
- Priority

The filtering/searching should preferably be handled through API query parameters rather than downloading all records and filtering everything only on the frontend.

9. Pagination

Task lists should support pagination.

Example:

```
← Previous     1  2  3  4     Next →
```

The frontend should use the pagination information returned by the backend API.

10. Admin Dashboard

Create a separate admin dashboard.

The admin dashboard should display:

- Total users
- Total tasks
- Pending tasks
- In-progress tasks
- Completed tasks
- High-priority tasks

Use suitable cards/charts/visualizations to present statistics clearly.

11. User Management

Admin should have a user management page.

Display:

- User name
- Email
- Role
- Number of tasks
- Created date
- Actions

Admin should be able to:

- View user details
- Manage users

- Delete users where permitted

Include search and pagination where appropriate.

12. Admin Task Management

Admin should have access to all tasks.

The admin should be able to:

- View all tasks
- Search tasks
- Filter tasks
- View task details
- Create tasks
- Assign tasks
- Update tasks
- Delete tasks

The admin should be able to easily identify:

- Task status
- Priority
- Assigned user
- Due date

13. Navigation

Create a proper application layout with navigation.

Normal User

```
Dashboard
Tasks
```

```
Profile
Logout
```

Admin

```
Dashboard
Tasks
Users
Profile
Logout
```

The navigation should change according to the logged-in user's role.

14. Responsive Design

The application should be fully responsive.

It should work properly on:

- Desktop
- Laptop
- Tablet
- Mobile

The dashboard, tables, forms, navigation and task cards should adapt to smaller screens.

15. Loading & Empty States

The application should properly handle loading states.

For example:

```
Loading tasks...
```

Instead of showing a blank screen.

Also implement useful empty states.

Example:

```
No tasks found.

Create your first task to get started.
```

16. Error Handling

The frontend should properly handle API errors.

Examples:

- Invalid login
- Session expired
- Unauthorized access
- Task not found
- User not found
- Validation errors
- Network errors
- Server errors

Show clear and user-friendly messages.

Avoid displaying raw backend errors directly to users.

17. Notifications

Use toast/snackbar notifications for important actions.

Examples:

```
Task created successfully
Task updated successfully
Task deleted successfully
Task assigned successfully
Something went wrong
```

18. Profile Page

Create a simple profile page.

Display:

- Name
- Email
- Role
- Account creation date

The password should never be displayed.

19. API Integration

The frontend must integrate with the provided backend APIs.

The frontend should not use hardcoded task/user data.

All important data should come from the backend.

Example flow:

```
Next.js Frontend
       ↓
API Request
       ↓
Express Backend
       ↓
MongoDB
       ↓
API Response
       ↓
Next.js UI
```

API base URL should be stored in environment variables.

Example:

```
NEXT_PUBLIC_API_URL=
```

20. Authentication State

The application should maintain the logged-in user's authentication state.

The frontend should handle:

- Login
- Logout
- Authentication token
- User information
- Role
- Protected routes
- Session expiration

Do not expose sensitive information unnecessarily on the client.

21. UI Components

Create reusable components instead of duplicating UI code.

For example:

```
components/
├── Button
├── Input
├── Modal
├── Navbar
├── Sidebar
├── TaskCard
├── TaskTable
├── StatusBadge
├── PriorityBadge
├── Pagination
├── Loading
└── EmptyState
```

The exact structure can be different, but the project should follow reusable-component principles.

22. Suggested Pages

The application should contain at least:

```
/login
/register

/dashboard

/tasks
```

```
/tasks/:id
/tasks/create
/tasks/:id/edit

/profile

/admin
/admin/users
/admin/tasks
```

The routing structure can be modified according to the candidate's implementation.

23. UI/UX Requirements

The interface should be:

- Modern
- Clean
- Responsive
- Easy to understand
- Consistent

Use consistent:

- Typography
- Spacing
- Buttons
- Forms
- Colors
- Status indicators
- Error messages

Avoid unnecessarily complicated designs.

24. Code Quality

The project should have:

- Clean component structure
- Reusable components
- Proper API service structure
- Proper error handling
- No unnecessary duplicate code
- No hardcoded API URLs
- No hardcoded user/task data
- Meaningful variable and function names

25. Optional / Bonus Features

The following features are optional:

- Dark mode
- Forgot password UI
- Email verification UI
- Task activity timeline
- Drag-and-drop task status
- Calendar view
- Advanced dashboard charts
- Real-time task updates using Socket.IO
- Skeleton loaders
- Unit/component testing

26. Expected User Flow

Normal User

```
Register
   ↓
Login
   ↓
Dashboard
   ↓
View Tasks
   ↓
Search / Filter
   ↓
Open Task
   ↓
Update Status
   ↓
Task Completed
```

Admin

```
Login
   ↓
Admin Dashboard
   ↓
View Statistics
   ↓
Manage Users
   ↓
Create Task
   ↓
Assign User
   ↓
Task Created
   ↓
Assigned User Receives Email
```

```
   ↓
Monitor Task
```

27. Expected Timeline

Duration: 2–3 days

The candidate should complete all major/core features within the given timeline.

Optional features can be implemented after completing the core requirements.

Final Requirement

The final application should be a complete **Task Management System frontend** connected to the backend API.

Users should be able to login and manage their tasks, while admins should be able to manage users, tasks, assignments and dashboard statistics.

The focus should be on **clean UI, responsive design, proper API integration, authentication, role-based access, reusable components and good frontend architecture**.